



PERTEMUAN KE-3 · STI5134

Memory, Storage, dan File System

Mata Kuliah: Layanan dan Sistem Virtual — Topik utama: Memory and Storage, File System, Distributed File System (DFS), Hadoop Distributed File System (HDFS), dan Google File System (GFS).

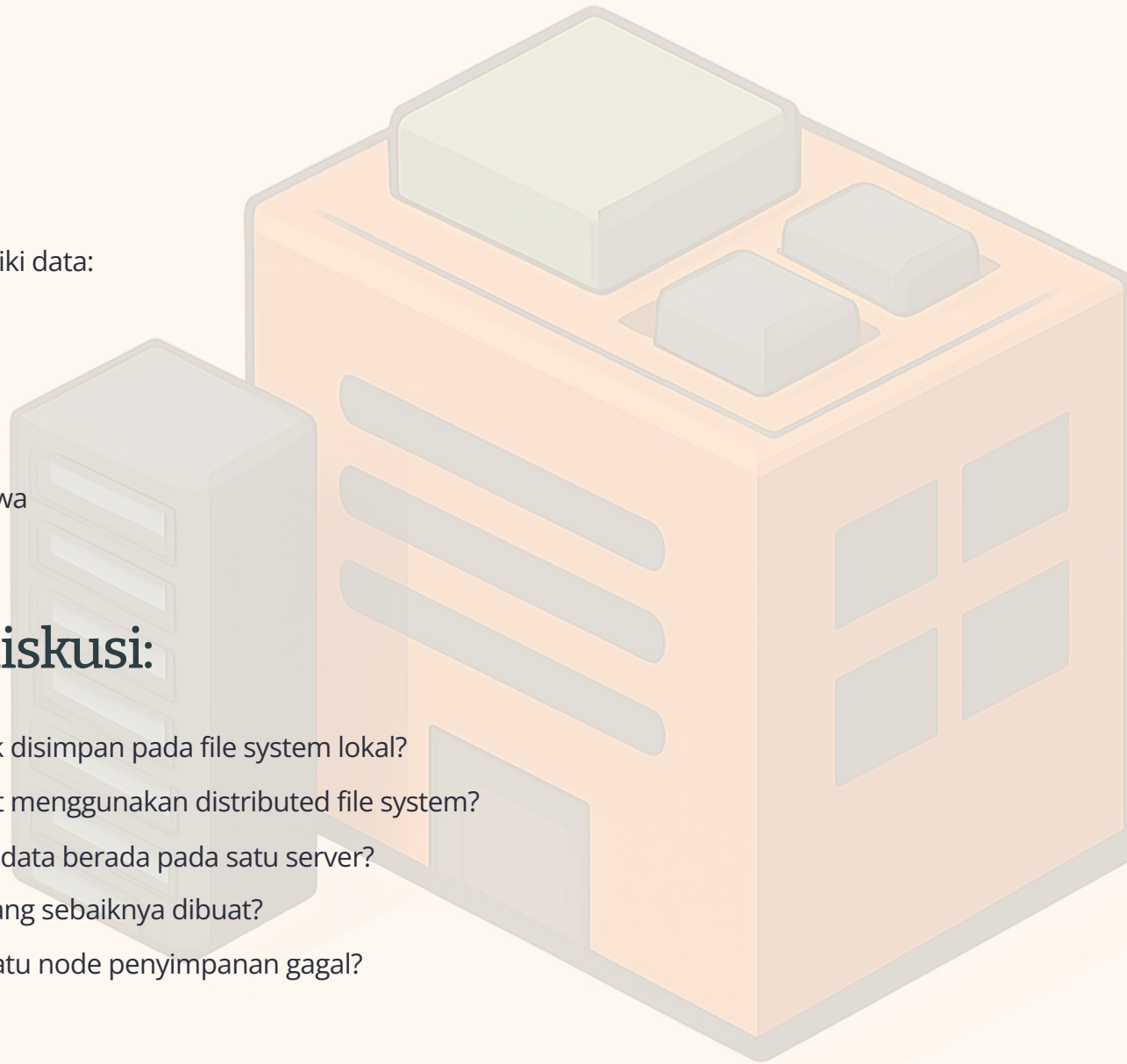
Studi Kasus

Sebuah universitas memiliki data:

- Video pembelajaran
- Data akademik
- Log sistem
- Berkas tugas mahasiswa
- Data penelitian

Pertanyaan diskusi:

1. Data mana yang cocok disimpan pada file system lokal?
2. Data mana yang dapat menggunakan distributed file system?
3. Apa risiko jika seluruh data berada pada satu server?
4. Berapa salinan data yang sebaiknya dibuat?
5. Apa yang terjadi jika satu node penyimpanan gagal?



Pengertian Hadoop Distributed File System (HDFS)

Hadoop Distributed File System (HDFS) merupakan singkatan dari Hadoop Distributed File System.

Hadoop Distributed File System (HDFS) dirancang untuk:

Menyimpan file berukuran besar.

Menyebarkan data ke banyak komputer.

Memberikan toleransi terhadap kegagalan.

Mendukung pemrosesan data berskala besar.

Berjalan pada perangkat keras yang relatif umum.

Hadoop Distributed File System (HDFS) lebih cocok untuk file besar dan pemrosesan berurutan dibandingkan file kecil yang sering diubah.



Metadata File

Selain isi file, sistem operasi juga menyimpan metadata.

Metadata dapat berisi:

- Nama file
- Ukuran file
- Jenis file
- Pemilik file
- Hak akses
- Waktu pembuatan
- Waktu perubahan
- Waktu terakhir diakses
- Lokasi blok data

Contoh:

Isi dokumen merupakan data, sedangkan ukuran, pemilik, dan waktu perubahan merupakan metadata.

Apa yang Akan Dipelajari?

Setelah mengikuti pertemuan ini, mahasiswa mampu:

01

Memori vs. Penyimpanan

Menjelaskan perbedaan fungsi, kecepatan, dan ketahanan data antara memori dan penyimpanan.

02

Sistem Berkas

Menjelaskan fungsi sistem berkas dan cara sistem operasi menyimpan serta menemukan file.

03

Distributed File System (DFS)

Menjelaskan konsep penyimpanan terdistribusi dan tantangan yang dihadapinya.

04

Arsitektur Hadoop Distributed File System (HDFS)

Menjelaskan komponen NameNode (pengelola metadata), DataNode (penyimpan data), mekanisme blok, dan replikasi otomatis.

05

Arsitektur Google File System (GFS)

Menjelaskan komponen Master dan Chunkserver serta membandingkannya dengan Hadoop Distributed File System (HDFS).

Aktivitas Information Search

Mahasiswa mencari informasi dari sumber terpercaya mengenai salah satu topik:

- File system
- Inode
- Distributed File System (DFS)
- Hadoop Distributed File System (HDFS)
- NameNode (pengelola metadata) dan DataNode (penyimpan data)
- Replikasi Hadoop Distributed File System (HDFS)
- Google File System (GFS)
- Perbandingan Google File System (GFS) dan Hadoop Distributed File System (HDFS)

Hal yang harus ditemukan:

- Pengertian konsep.
- Cara kerja.
- Komponen utama.
- Kelebihan.
- Keterbatasan.
- Contoh penerapan.

Arsitektur Hadoop Distributed File System (HDFS)

Hadoop Distributed File System (HDFS) memiliki dua komponen utama.

NameNode (pengelola metadata)

Bertugas mengelola metadata:

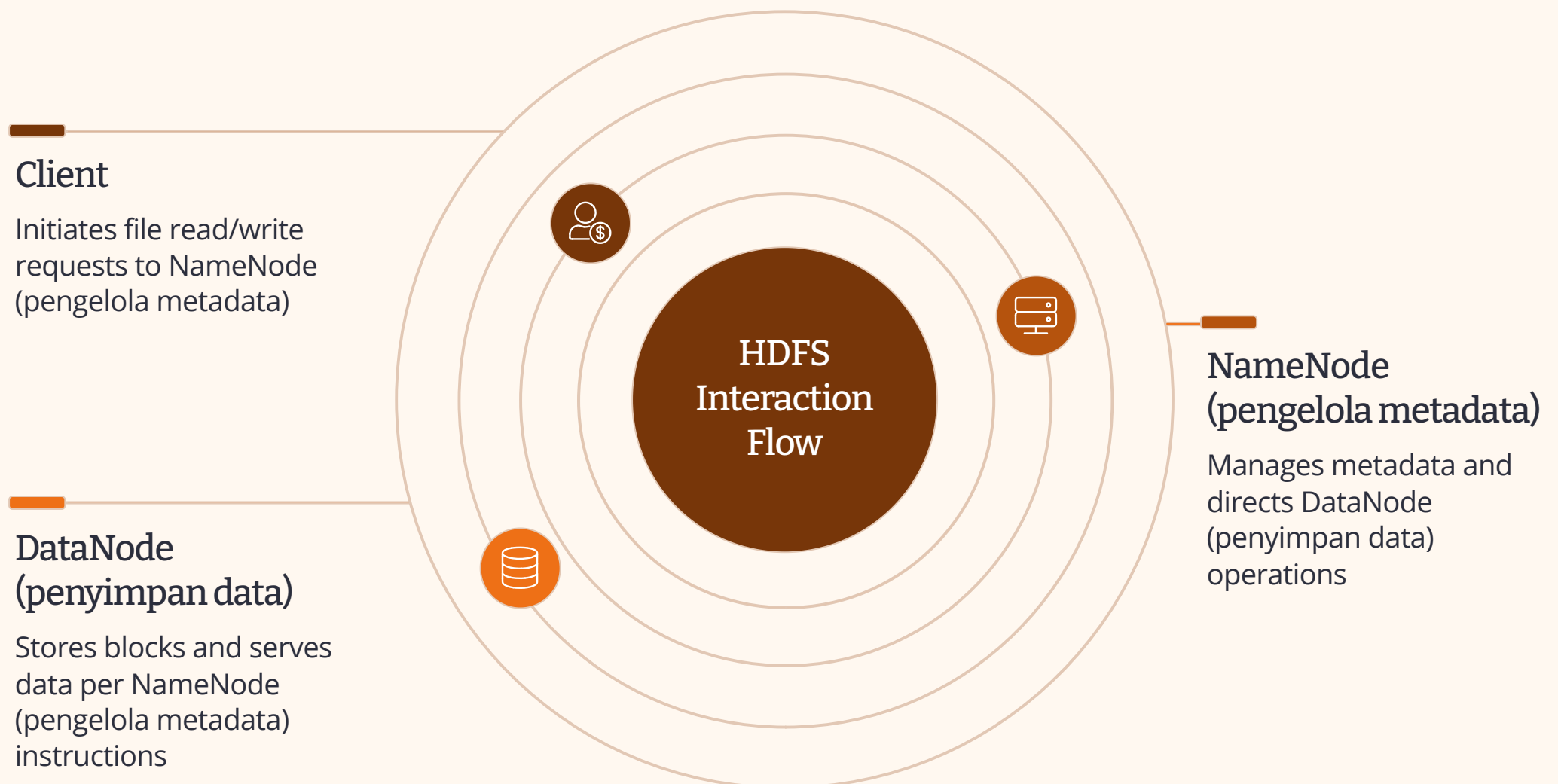
- Nama file
- Struktur direktori
- Permission
- Lokasi blok
- Informasi DataNode (penyimpan data)

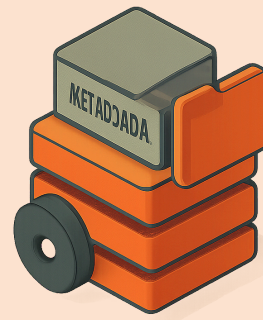
DataNode (penyimpan data)

Bertugas:

- Menyimpan blok data.
- Membaca dan mengirimkan blok.
- Melaporkan kondisi kepada NameNode (pengelola metadata).
- Membuat atau menghapus replika sesuai perintah.

Struktur sederhana:





Inode pada Linux

Inode adalah struktur data yang menyimpan metadata suatu file pada beberapa file system Linux.

Inode dapat menyimpan:

Jenis file	Ukuran file
User pemilik	Group pemilik
Permission	Timestamp
Jumlah hard link	Penunjuk lokasi blok data

Catatan penting:

Nama file tidak disimpan langsung di dalam inode. Direktori menghubungkan nama file dengan nomor inode.

Ukuran inode bergantung pada file system dan konfigurasinya. Pada ext4 (Extended File System 4), ukuran yang umum digunakan adalah 256 byte, meskipun ukuran lain dapat digunakan.

Mengapa File Sistem Itu Penting?

File tidak sekadar diletakkan pada hard disk. Sistem operasi memerlukan aturan untuk memberi nama, mencatat lokasi, mengatur hak akses, dan menemukan kembali file tersebut.

File Hilang

Mengapa file yang sudah disimpan kadang tidak ditemukan?

RAM (Random Access Memory) Penuh

Mengapa laptop kehabisan RAM (Random Access Memory) padahal penyimpanan masih tersedia?

Disk Rusak

Apa yang terjadi jika hard disk server rusak?

Data Besar

Bagaimana perusahaan menyimpan file berukuran sangat besar?

Google

Bagaimana Google menyimpan data dari jutaan pengguna?

Tugas Pertemuan 3

Membuat ringkasan minimal lima paragraf.

Struktur ringkasan:

Paragraf 1:

Perbedaan memory dan storage.

Paragraf 2:

Pengertian dan fungsi file system.

Paragraf 3:

Pengertian distributed file system.

Paragraf 4:

Arsitektur dan cara kerja Hadoop Distributed File System (HDFS).

Paragraf 5:

Google File System (GFS) dan perbandingannya dengan Hadoop Distributed File System (HDFS).

Ketentuan:

- Minimal lima paragraf.
- Menggunakan bahasa sendiri.
- Mencantumkan sumber.
- Tidak menyalin seluruh isi artikel.
- Dikumpulkan sesuai jadwal dosen.

Analogi Sistem Kurir

Hadoop Distributed File System (HDFS) dapat dianalogikan seperti perusahaan kurir.

Sistem Kurir	Hadoop Distributed File System (HDFS)
Kantor pusat	NameNode (pengelola metadata)
Gudang cabang	DataNode (penyimpan data)
Daftar lokasi paket	Metadata
Paket	Blok data
Nomor resi	Identitas blok
Salinan paket	Replikasi
Pelanggan	Client

Kantor pusat mengetahui lokasi paket, tetapi paket fisik berada di gudang cabang.



Analogi Kartu Tanda Penduduk File

Inode dapat dianalogikan sebagai Kartu Tanda Penduduk (KTP) sebuah file.

Kartu Tanda Penduduk (KTP) Penduduk	Inode File
Nomor identitas	Nomor inode
Nama pemilik	User pemilik
Informasi pribadi	Metadata
Alamat	Penunjuk lokasi data
Masa perubahan data	Timestamp

Nama file berfungsi seperti nama panggilan, sedangkan inode menjadi identitas internal yang digunakan sistem operasi.

Dua nama file bahkan dapat menunjuk pada inode yang sama melalui mekanisme hard link.



Memory dan Storage: Analogi Dapur Restoran

Komponen komputer dapat dianalogikan dengan dapur restoran. Bahan yang sedang digunakan diletakkan di meja agar mudah dijangkau, sementara bahan yang belum digunakan tetap di gudang.

Jika meja terlalu kecil, koki harus sering bolak-balik ke gudang — pekerjaan menjadi lebih lambat. Ini seperti RAM (Random Access Memory) yang penuh: sistem terpaksa menggunakan swap yang jauh lebih lambat.

- ❏ Menambah kapasitas storage tidak otomatis menyelesaikan kekurangan RAM (Random Access Memory). Memory dan storage memiliki fungsi yang berbeda.

Analogi Dapur Restoran

Dapur Restoran

Meja memasak

Koki

Gudang bahan

Buku daftar bahan

Pesanan pelanggan

Sistem Komputer

RAM (Random Access Memory)

CPU

Storage

File system

Program berjalan

Proses Menyimpan File di Hadoop Distributed File System (HDFS)

Misalnya pengguna menyimpan file berukuran besar.

Prosesnya:

01

Client menghubungi NameNode (pengelola metadata).

02

NameNode (pengelola metadata) memeriksa nama, direktori, dan hak akses.

03

File dibagi menjadi beberapa blok.

04

NameNode (pengelola metadata) menentukan DataNode (penyimpan data) tujuan.

05

Client mengirim blok ke DataNode (penyimpan data).

06

DataNode (penyimpan data) membuat replika pada node lain.

07

NameNode (pengelola metadata) mencatat lokasi semua blok.

08

Proses selesai setelah blok dan replika berhasil disimpan.

Jenis File System yang Umum Digunakan

File System	Sistem Operasi	Karakteristik
FAT32 (File Allocation Table 32-bit)	Windows dan perangkat eksternal	Kompatibel, tetapi ukuran file terbatas
exFAT (Extended File Allocation Table)	Berbagai sistem operasi	Cocok untuk flashdisk berkapasitas besar
NTFS (New Technology File System)	Windows	Permission, jurnal, dan file berukuran besar
ext4 (Extended File System 4)	Linux	Stabil dan umum digunakan
APFS (Apple File System)	macOS	Dirancang untuk perangkat Apple
XFS (XFS File System)	Linux dan server	Cocok untuk file serta storage besar

Pemilihan file system harus mempertimbangkan sistem operasi, ukuran data, keamanan, dan kebutuhan penggunaan.

Perbandingan Memory dan Storage

Aspek	Memory	Storage
Contoh	RAM (Random Access Memory) dan cache	SSD (Solid State Drive) dan HDD (Hard Disk Drive)
Fungsi	Menampung data yang sedang diproses	Menyimpan data jangka panjang
Kecepatan	Sangat cepat	Lebih lambat
Kapasitas	Relatif kecil	Relatif besar
Ketahanan data	Hilang saat listrik mati	Tetap tersimpan
Harga per GB	Lebih mahal	Lebih murah

Selain RAM (Random Access Memory), sistem operasi juga menggunakan **swap** — area storage yang digunakan ketika RAM (Random Access Memory) hampir penuh. Data yang jarang digunakan dipindahkan ke swap untuk membebaskan ruang RAM (Random Access Memory). Namun, swap jauh lebih lambat daripada RAM (Random Access Memory). Jika RAM (Random Access Memory) dan swap keduanya habis, **OOM Killer (Out of Memory Killer)** pada Linux akan menghentikan proses tertentu agar sistem tetap berjalan.

Latihan Evaluasi

Uji pemahaman Anda tentang konsep-konsep kunci dalam materi ini:

- 1 Apa perbedaan memory dan storage?
- 2 Mengapa swap lebih lambat daripada RAM (Random Access Memory)?
- 3 Apa fungsi file system?
- 4 Apa fungsi inode pada Linux?
- 5 Mengapa Distributed File System (DFS) membutuhkan replikasi?
- 6 Apa perbedaan NameNode (pengelola metadata) dan DataNode (penyimpan data)?
- 7 Mengapa Hadoop Distributed File System (HDFS) menggunakan blok berukuran besar?
- 8 Bagaimana Hadoop Distributed File System (HDFS) menangani DataNode (penyimpan data) yang gagal?
- 9 Apa persamaan Google File System (GFS) dan Hadoop Distributed File System (HDFS)?
- 10 Apa kelemahan menyimpan seluruh data pada satu server?



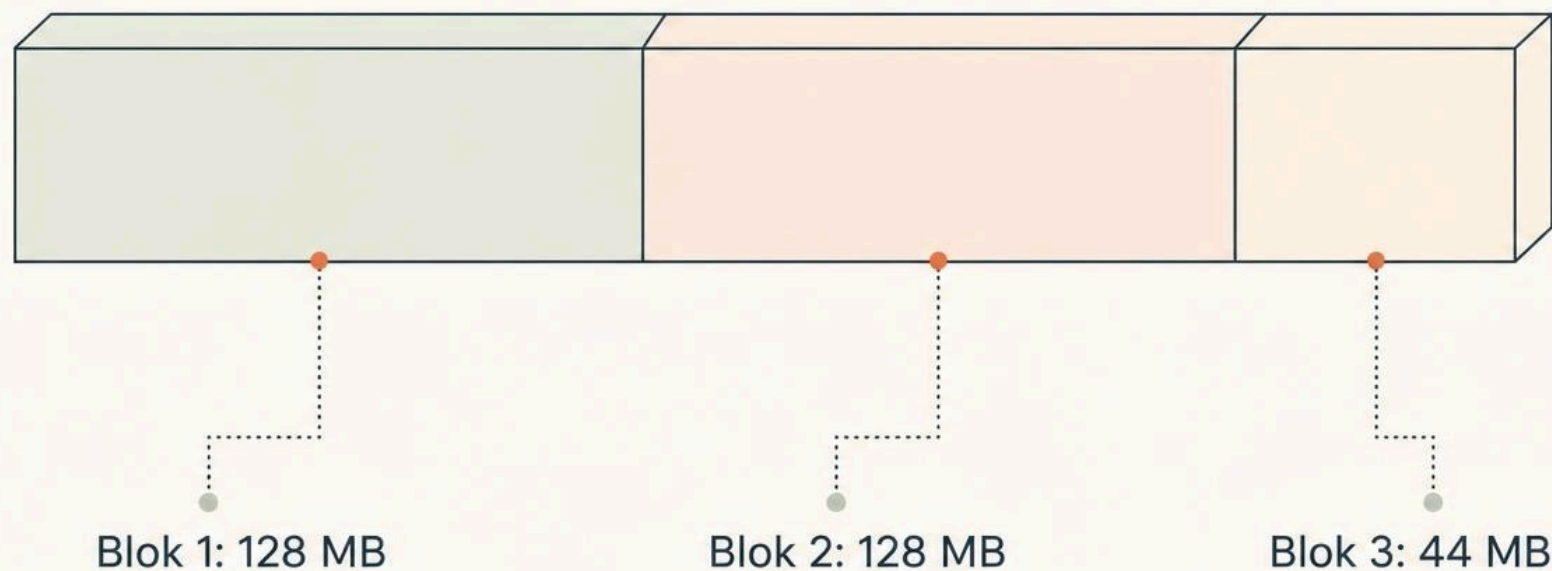
Mengapa Blok Hadoop Distributed File System (HDFS) Besar?

Hadoop Distributed File System (HDFS) umumnya menggunakan ukuran blok besar, misalnya 128 MB.

Alasannya:

- Mengurangi jumlah metadata.
- Mengurangi frekuensi pencarian blok.
- Meningkatkan efisiensi transfer data besar.
- Mendukung pembacaan berurutan.
- Mempermudah pemrosesan data terdistribusi.

Contoh:





FILE SYSTEM

Journaling File System

Journaling merupakan mekanisme pencatatan perubahan sebelum perubahan utama dilakukan.

Alur sederhananya:

1

Sistem mencatat rencana perubahan pada journal.

2

Sistem melakukan perubahan pada file system.

3

Sistem menandai bahwa perubahan telah selesai.

Jika listrik mati, journal membantu proses pemulihan.

Contoh file system yang mendukung journaling:

- NTFS (New Technology File System)
- ext4 (Extended File System 4)
- XFS (XFS File System)

Manfaat utama:

- 📄 Journaling mengurangi risiko kerusakan struktur file system setelah sistem mati secara tidak normal.

Pengertian File System

File system adalah aturan dan struktur yang digunakan sistem operasi untuk menyimpan, menamai, mengelompokkan, dan menemukan kembali file. Tanpa file system, storage hanya berisi kumpulan data biner yang sulit dikenali.



Penamaan & Organisasi

Memberikan nama file dan mengelompokkannya dalam direktori secara hierarkis.



Pelacakan Lokasi

Mencatat lokasi fisik data pada storage menggunakan inode atau metadata.



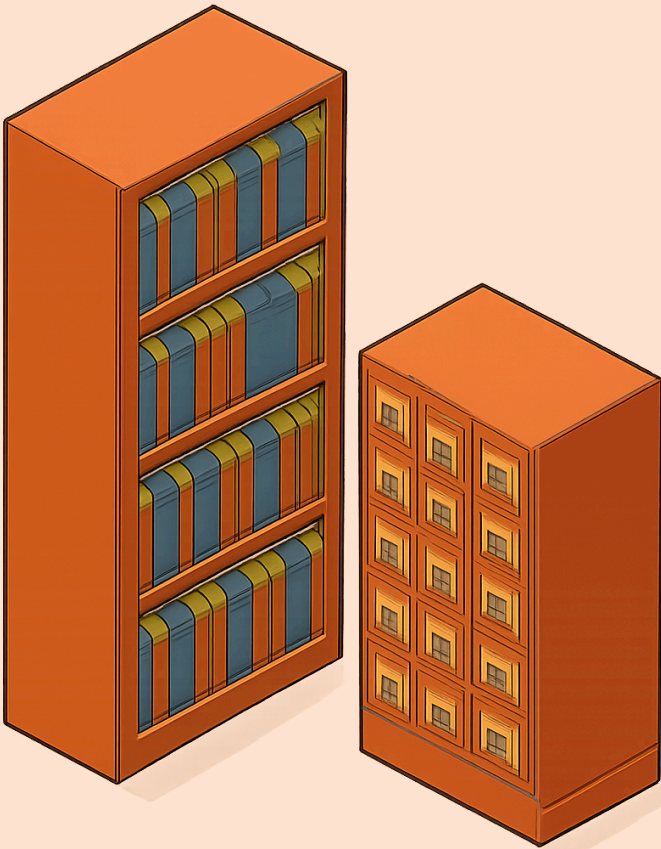
Hak Akses

Mengatur pemilik file, permission, dan siapa yang dapat membaca atau menulis.



Deteksi Kerusakan

Mendeteksi kerusakan tertentu dan memulihkan struktur file system melalui journaling.



Ringkasan



RAM (Random Access Memory) menyimpan data yang sedang diproses.



Storage menyimpan data dalam jangka panjang.



File system mengatur nama, lokasi, metadata, dan akses file.



Inode menyimpan metadata file pada beberapa file system Linux.



Distributed File System (DFS) menyebarkan data ke beberapa komputer.



Hadoop Distributed File System (HDFS) menggunakan NameNode (pengelola metadata) dan DataNode (penyimpan data).



Hadoop Distributed File System (HDFS) membagi file menjadi blok dan membuat replika.



Google File System (GFS) menggunakan Master dan Chunkserver.



Replikasi membantu sistem bertahan ketika perangkat mengalami kegagalan.

Replikasi dan Auto-Repair Hadoop Distributed File System (HDFS)

Setiap blok dapat disimpan pada beberapa DataNode (penyimpan data).



Replikasi Awal

Blok A disimpan pada: DataNode (penyimpan data) 1, DataNode (penyimpan data) 2, DataNode (penyimpan data) 3.



DataNode Gagal

Jika DataNode (penyimpan data) 2 gagal, NameNode (pengelola metadata) tidak menerima heartbeat.



Deteksi Masalah

NameNode (pengelola metadata) menganggap node bermasalah.



Data Tetap Tersedia

Masih tersedia dua salinan Blok A yang aktif.



Re-replikasi

NameNode (pengelola metadata) memerintahkan pembuatan replika baru di DataNode (penyimpan data) lain.



Jumlah Salinan Kembali Normal

Jumlah salinan kembali menjadi tiga.

Mekanisme ini disebut re-replication atau pemulihan otomatis.

OverlayFS dan Container

OverlayFS menggabungkan beberapa lapisan direktori menjadi satu tampilan.

Analogi:

OverlayFS seperti meletakkan plastik transparan di atas sebuah peta. Peta dasar tidak berubah. Informasi tambahan ditulis pada lapisan transparan.

Pada Docker:



Image Lapisan Dasar

Image menjadi lapisan dasar yang hanya dapat dibaca.



Container Lapisan Tulis

Container menambahkan lapisan yang dapat ditulis.



Perubahan Terpisah

Perubahan container tidak langsung mengubah image dasar.



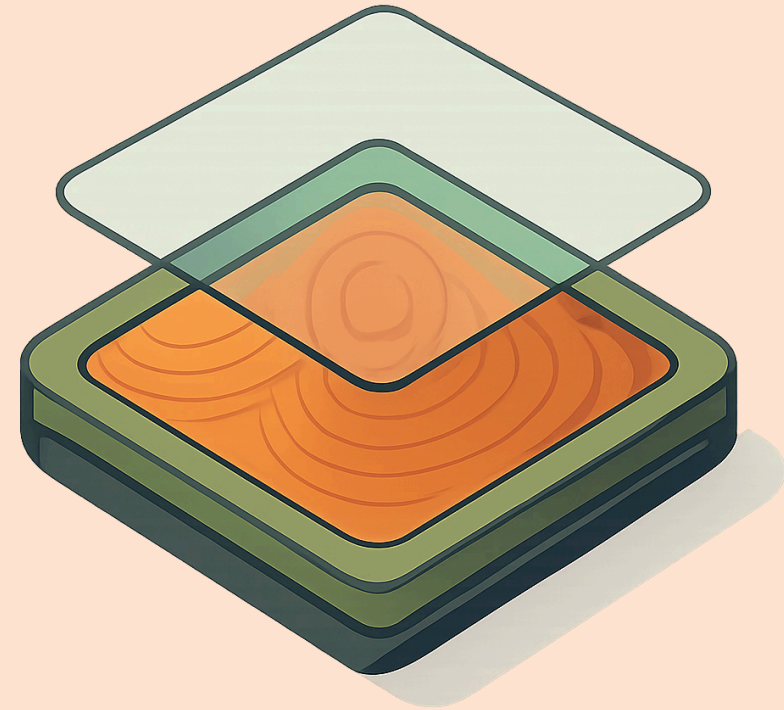
Berbagi Image

Beberapa container dapat menggunakan lapisan image yang sama.



Hemat Penyimpanan

Ini membantu menghemat ruang penyimpanan.



Inode dan Metadata File

Analogi Kartu Tanda Penduduk File

Inode dapat dianalogikan sebagai **Kartu Tanda Penduduk sebuah file**. Nama file berfungsi seperti nama panggilan, sedangkan inode menjadi identitas internal yang digunakan sistem operasi.

Kartu Tanda Penduduk Penduduk → Inode File

Nomor identitas = Nomor inode

Nama pemilik → User pemilik

Informasi pribadi = Metadata

Alamat → Penunjuk lokasi data

Masa perubahan = Timestamp

Isi Inode pada Linux

- Jenis dan ukuran file
- User dan group pemilik
- Permission (rwx)
- Timestamp (buat, ubah, akses)
- Jumlah hard link
- Penunjuk lokasi blok data

Catatan: Nama file **tidak** disimpan di dalam inode. Direktori menghubungkan nama file dengan nomor inode.

Google File System (GFS)

Google File System (GFS) merupakan Distributed File System (DFS) yang dikembangkan Google.

Google File System (GFS) dirancang untuk:



Menyimpan data dalam skala besar.



Berjalan pada banyak server.



Menghadapi kegagalan perangkat.



Mendukung file berukuran sangat besar.



Mendukung beban kerja Google.



Melakukan pemulihan secara otomatis.

Makalah Google File System (GFS) dipublikasikan pada Search of Operating Systems and Programming (SOSP) tahun 2003 dan menjadi salah satu inspirasi pengembangan Hadoop Distributed File System (HDFS).

Keterbatasan File System Lokal

File system lokal bekerja dengan baik pada satu komputer, tetapi memiliki keterbatasan:

Kapasitas terbatas pada satu perangkat.

Kerusakan disk dapat menyebabkan kehilangan data.

Sulit melayani pengguna dalam jumlah besar.

Tidak mudah diperluas.

Menjadi titik kegagalan tunggal.

Kinerja terbatas pada satu server.

Kebutuhan data besar mendorong penggunaan distributed file system.

Dari File System Lokal ke Distributed File System (DFS)

Keterbatasan File System Lokal

- Kapasitas terbatas pada satu perangkat
- Kerusakan disk dapat menghilangkan data
- Sulit melayani banyak pengguna sekaligus
- Menjadi titik kegagalan tunggal (single point of failure)

Solusi: Distributed File System (DFS)

Distributed File System (DFS) membagi data ke beberapa komputer yang terhubung melalui jaringan, namun pengguna tetap melihatnya sebagai satu sistem penyimpanan.

Skalabilitas

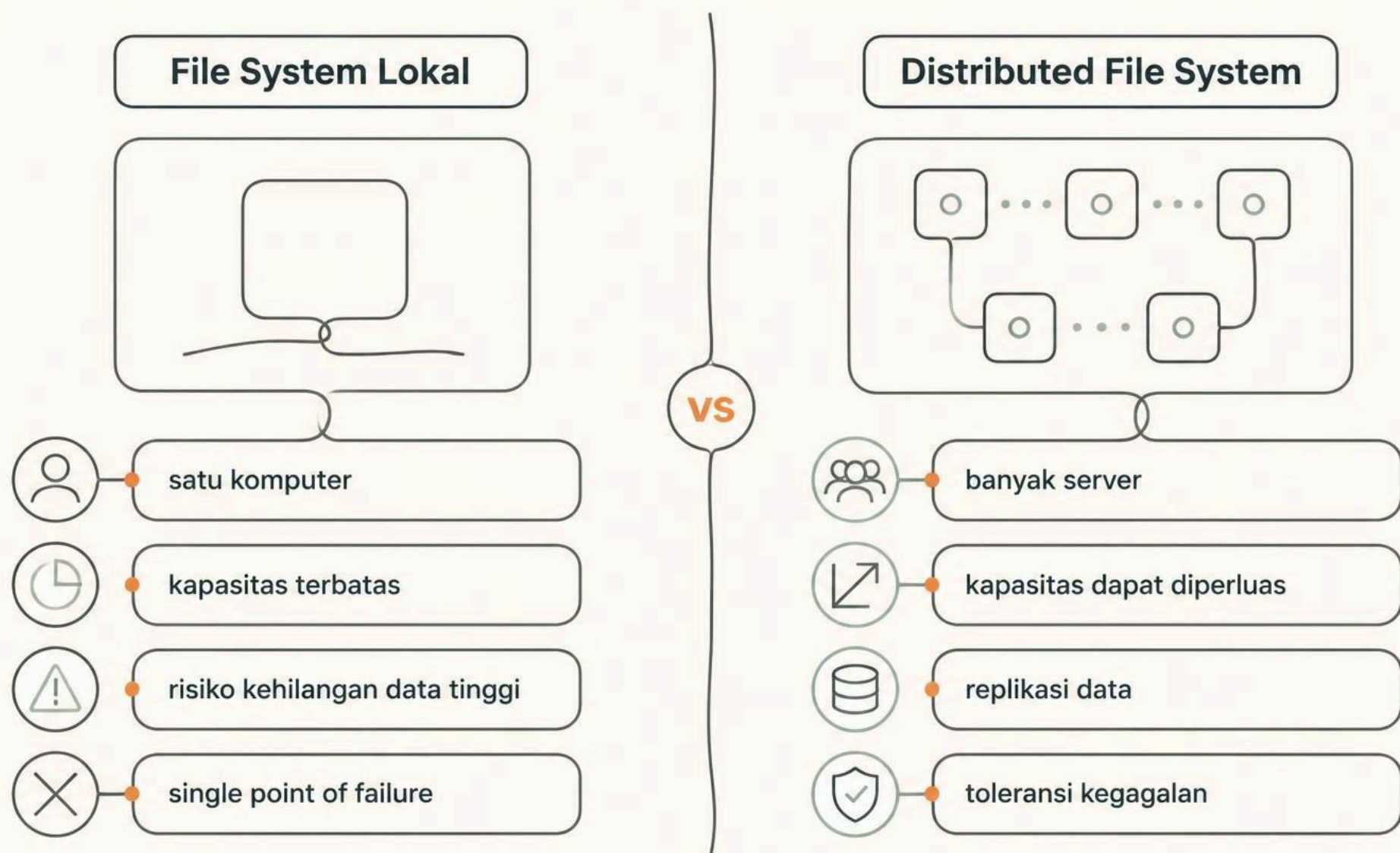
Kapasitas ditambah dengan server baru

Replikasi

Data disalin ke beberapa server

Toleransi Gagal

Kerusakan satu server tidak menghilangkan data



Arsitektur Google File System (GFS)

Google File System (GFS) memiliki beberapa komponen utama.



Google File System (GFS) Master

- Namespace
- Metadata
- Lokasi chunk
- Replikasi
- Pengaturan akses



Chunkserver

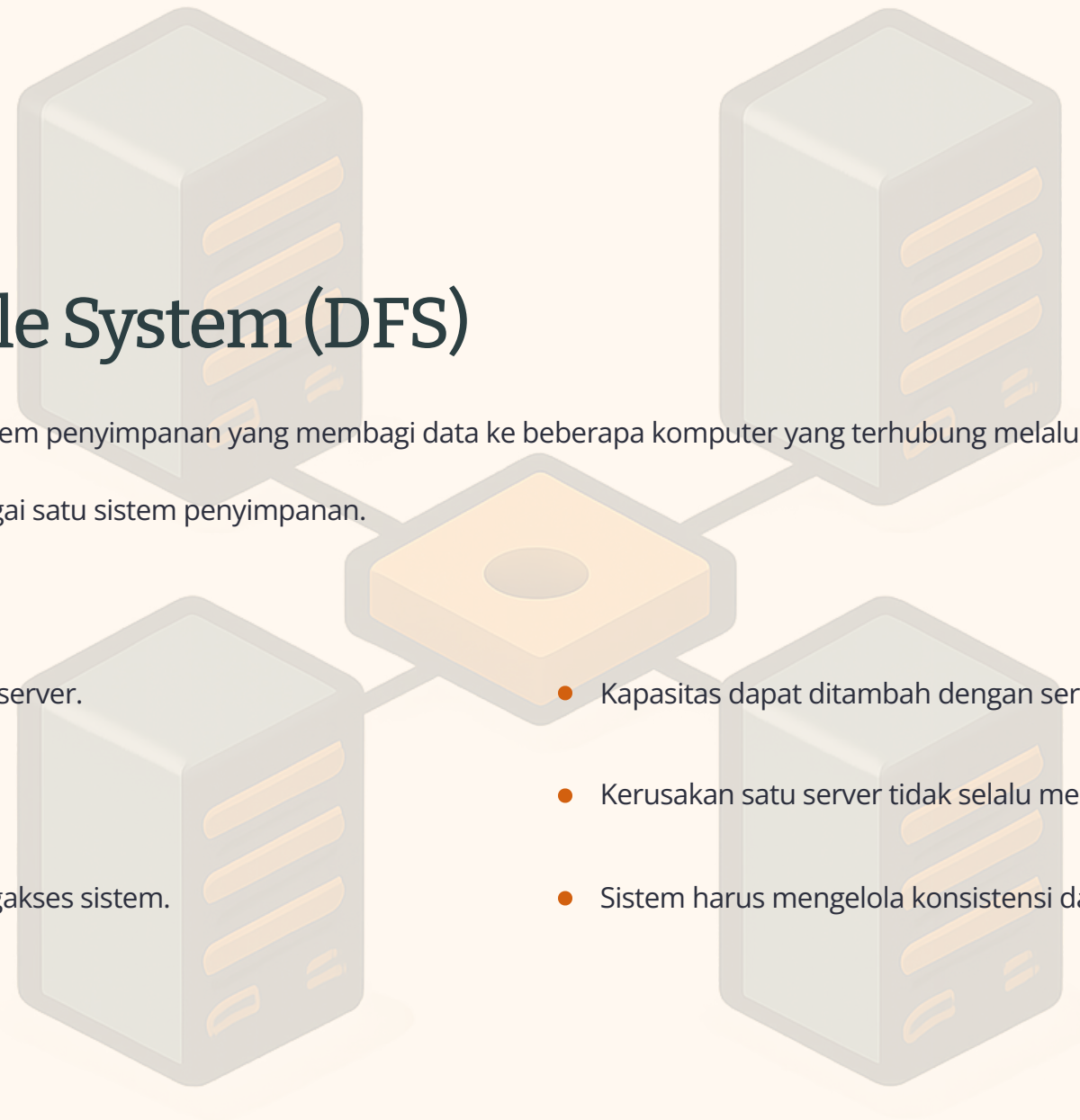
Menyimpan bagian file yang disebut chunk.



Client

Meminta metadata kepada Master, kemudian berkomunikasi langsung dengan Chunkserver untuk mengakses data.

Master tidak menjadi jalur utama transfer isi file.



DISTRIBUTED FILE SYSTEM (DFS)

Distributed File System (DFS)

Distributed file system adalah sistem penyimpanan yang membagi data ke beberapa komputer yang terhubung melalui jaringan.

Pengguna tetap melihatnya sebagai satu sistem penyimpanan.

Karakteristiknya:

- Data tersebar pada beberapa server.
- Data dapat direplikasi.
- Banyak pengguna dapat mengakses sistem.
- Kapasitas dapat ditambah dengan server baru.
- Kerusakan satu server tidak selalu menghilangkan data.
- Sistem harus mengelola konsistensi data.

Hadoop Distributed File System (HDFS)

Hadoop Distributed File System (HDFS) dirancang untuk menyimpan file berukuran besar, menyebarkan data ke banyak komputer, dan memberikan toleransi terhadap kegagalan. Hadoop Distributed File System (HDFS) lebih cocok untuk file besar dan pemrosesan berurutan.

1

NameNode (pengelola metadata)

Mengelola metadata: nama file, struktur direktori, lokasi blok, dan informasi DataNode (penyimpan data).

2

Pembagian Blok

File dibagi menjadi blok berukuran besar (128 MB). Contoh: file 300 MB → 3 blok.

3

DataNode (penyimpan data)

Menyimpan blok data, membaca dan mengirimkan blok, serta melaporkan kondisi ke NameNode (pengelola metadata).

4

Replikasi

Setiap blok direplikasi ke beberapa DataNode (penyimpan data) (faktor 3). Jika node gagal, NameNode (pengelola metadata) memerintahkan pembuatan replika baru secara otomatis.



Perbandingan Google File System (GFS) dan Hadoop Distributed File System (HDFS)

Aspek	Google File System (GFS)	Hadoop Distributed File System (HDFS)
Pengembang	Google	Apache Hadoop
Istilah bagian file	Chunk	Block
Pengelola metadata	Master	NameNode (pengelola metadata)
Penyimpan data	Chunkserver	DataNode (penyimpan data)
Fokus awal	Infrastruktur internal Google	Ekosistem Hadoop
Toleransi kegagalan	Replikasi	Replikasi
Ukuran bagian data	Besar	Besar
Penggunaan	Internal Google	Terbuka dan banyak digunakan

Kesimpulan:

Google File System (GFS) dan Hadoop Distributed File System (HDFS) memiliki prinsip yang mirip, tetapi dikembangkan untuk lingkungan yang berbeda.

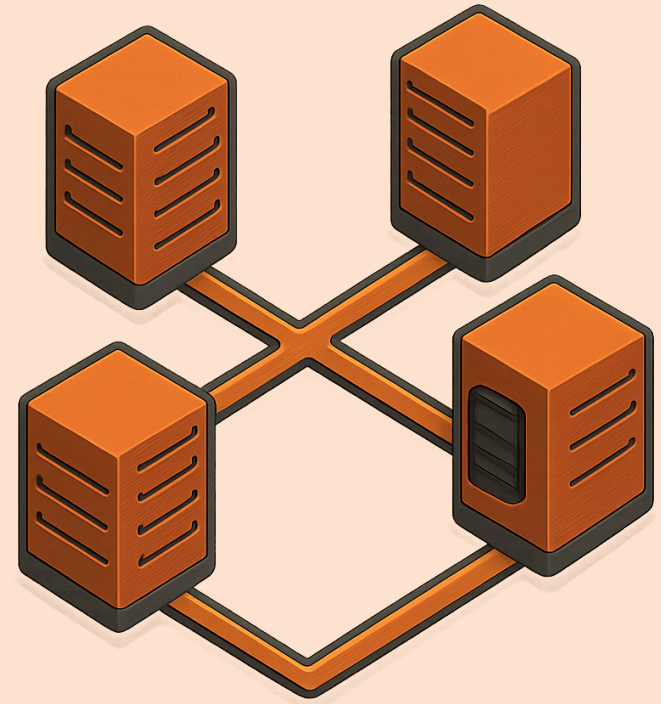
Analogi Perpustakaan Nasional

File system lokal seperti perpustakaan kecil di satu kota.

Distributed File System (DFS) seperti jaringan perpustakaan nasional.

Jika satu perpustakaan tidak memiliki buku, pengguna dapat memperoleh salinan dari cabang lain.

Perpustakaan Nasional	Distributed File System (DFS)
Cabang perpustakaan	Server penyimpanan
Salinan buku	Replikasi data
Katalog pusat	Metadata server
Pengiriman antarcabang	Transfer data melalui jaringan
Cabang yang tutup	Node gagal



Google File System (GFS) dan Perbandingan

Google File System (GFS)

Google File System (GFS) dikembangkan Google dan dipublikasikan di Symposium on Operating Systems Principles (SOSP) 2003. Sistem ini menjadi inspirasi utama Hadoop Distributed File System (HDFS).

- **GFS Master:** mengelola namespace, metadata, lokasi chunk, dan replikasi
- **Chunkserver:** menyimpan bagian file yang disebut chunk
- **Client:** meminta metadata ke Master, lalu mengakses data langsung dari Chunkserver

Perbandingan Google File System (GFS) dan Hadoop Distributed File System (HDFS)

Aspek	Google File System (GFS)	Hadoop Distributed File System (HDFS)
Pengembang	Google	Apache Hadoop
Metadata	Master	NameNode (pengelola metadata)
Penyimpan data	Chunkserver	DataNode (penyimpan data)
Pemulihan	Otomatis	Otomatis
Penggunaan	Internal Google	Terbuka, big data

- 📌 **Tugas Pertemuan 3 (Bobot 10%):** Buat ringkasan minimal 5 paragraf mencakup: (1) memory vs storage, (2) file system, (3) distributed file system (DFS), (4) arsitektur Hadoop Distributed File System (HDFS), (5) Google File System (GFS) dan perbandingannya. Gunakan bahasa sendiri dan cantumkan sumber.

File System Lokal, Hadoop Distributed File System (HDFS), dan Google File System (GFS)

Aspek	File System Lokal	Hadoop Distributed File System (HDFS)	Google File System (GFS)
Jumlah komputer	Satu	Banyak	Banyak
Skala data	Kecil sampai menengah	Sangat besar	Sangat besar
Metadata	Di komputer lokal	NameNode (pengelola metadata)	Master (pengelola metadata)
Penyimpanan data	Disk lokal	DataNode (penyimpan data)	Chunkserver (penyimpan chunk)
Replikasi	Tidak selalu tersedia	Tersedia	Tersedia
Pemulihan node	Umumnya manual	Otomatis	Otomatis
Pengguna utama	Komputer pribadi dan server biasa	Hadoop dan big data	Infrastruktur Google

Tantangan Distributed File System (DFS)

Tantangan	Masalah	Solusi Umum
Kegagalan perangkat	Server atau disk dapat rusak	Replikasi dan auto-repair
Kapasitas	Data terus bertambah	Menambah node
Kecepatan	Transfer jaringan lebih lambat	Memproses data dekat lokasi penyimpanan
Konsistensi	Banyak pengguna mengubah data	Aturan konsistensi
Keamanan	Data tersebar di banyak server	Autentikasi dan kontrol akses
Metadata	Jumlah file sangat banyak	Metadata server khusus

Struktur File System

Struktur file system umumnya berbentuk hierarki.

Contoh pada Linux:

```
/
├── home
│   ├── mahasiswa
│   │   ├── dokumen
│   │   └── tugas
├── etc
├── var
└── tmp
```

Keterangan:

- / merupakan direktori utama atau root.
- /home menyimpan data pengguna.
- /etc menyimpan konfigurasi sistem.
- /var menyimpan data yang sering berubah.
- /tmp menyimpan file sementara.

Rubrik Penilaian

Komponen	Bobot
Ketepatan konsep memory dan storage	15%
Ketepatan konsep file system	20%
Penjelasan Distributed File System (DFS)	20%
Penjelasan Hadoop Distributed File System (HDFS)	20%
Penjelasan Google File System (GFS)	15%
Penggunaan sumber dan kerapian	10%
Total	100%

Bobot pertemuan dalam Rencana Pembelajaran Semester (RPS) adalah 10%.